

NBA Foul Predictor

Imports

```
import numpy as np
import polars as pl
from pyprojroot.here import here

## To fix format of PDF output
pl.Config.set_tbl_cols(8);    ## show at most 8 columns
pl.Config.set_tbl_rows(10);   ## show at most 10 rows
```

Data Cleaning and File Merging

Combining the Referee Stats Files Into One Table

```
seasons = ['2017-2018', '2018-2019', '2019-2020', '2020-2021', '2021-2022', '2022-2023', '2023-2024', '2024-2025',
           '2025-2026']

refList = []
for s in seasons:
    fileName = f'nba_referee_stats_{s}.csv'
    if s == '2025-2026':
        fileName = 'nba_referee_stats_2025_2026.csv' ## uses an underscore instead of dash between years

    refSeason = pl.read_csv(here(f'Data/{fileName}'))

    ## find foul differential column
    diffCol = [c for c in refSeason.columns if 'FOUL DIFFERENTIAL' in c][0]

    refSeason = (refSeason
                 .rename({diffCol: 'foul_diff'})
                 .rename({'TYPE': 'ROLE'}, strict = False) ## older files say TYPE
                 .drop(['RANK', 'Unnamed: 13'], strict = False) ## empty columns
                 .with_columns(Season = pl.lit(s)) ## adds Season if missing
                 )
    refList.append(refSeason)

## Stack all seasons
refStats = pl.concat(refList, how = 'diagonal')

## Renaming columns
refStats = refStats.rename({
    'Season': 'season', 'REFEREE': 'referee', 'ROLE': 'role', 'GENDER': 'gender', 'EXPERIENCE (YEARS)': 'experience',
    'GAMES OFFICIATED': 'games', 'HOME TEAM WIN%': 'home_win_pct', 'HOME TEAM POINTS DIFFERENTIAL': 'home_pt_diff',
    'TOTAL POINTS PER GAME': 'points_per_game', 'CALLED FOULS PER GAME': 'fouls_per_game', 'FOUL% AGAINST ROAD TEAMS':
    'foul_pct_road', 'FOUL% AGAINST HOME TEAMS': 'foul_pct_home',
})

## Fixing percents
refStats = refStats.with_columns(
    pl.when(pl.col(c) > 1).then(pl.col(c) / 100).otherwise(pl.col(c)).alias(c)
    for c in ['foul_pct_road', 'foul_pct_home']
)

refStats
```

season	referee	role	experience	...	foul_pct_road	foul_pct_home	foul_diff	gender
str	str	str	i64	...	f64	f64	f64	str
"2017-2018"	"Ben Taylor"	"MAIN"	6	...	0.538	0.462	3.0	null
"2017-2018"	"Bennie Adams"	"MAIN"	22	...	0.498	0.502	-0.1	null
"2017-2018"	"Bill Kennedy"	"MAIN"	22	...	0.51	0.49	0.8	null
"2017-2018"	"Bill Spooner"	"MAIN"	28	...	0.513	0.488	1.0	null
"2017-2018"	"Brent Barnaky"	"MAIN"	8	...	0.493	0.507	-0.6	null
...	...	...	...	...	...	...	...	...
"2025-2026"	"Sean Corbin"	"CREW"	33	...	0.5	0.5	0.0	"MALE"
"2025-2026"	"Che Flores"	"CHIEF"	5	...	0.521	0.479	1.5	"FEMALE"
"2025-2026"	"Eric Lewis"	"CHIEF"	22	...	0.539	0.461	3.2	"MALE"
"2025-2026"	"Angel Kent"	"CREW"	1	...	0.447	0.553	-4.0	"FEMALE"
"2025-2026"	"Ben Taylor"	"CHIEF"	15	...	0.509	0.491	0.6	"MALE"

Combine CHIEF and CREW Rows

```

avgCols = ['home_win_pct', 'home_pt_diff', 'points_per_game', 'fouls_per_game', 'foul_pct_road', 'foul_pct_home',
'foul_diff']

refStats = (refStats
    .group_by(['season', 'referee']) ## referee rows for a season together
    .agg(
        experience = pl.col('experience').max(),
        gender = pl.col('gender').first(),
        games = pl.col('games').sum(), ## adding them
        *(((pl.col(c) * pl.col('games')).sum() / pl.col('games').sum()).alias(c) for c in avgCols) ## weighted average
        for every column in avgCols
    )
    .sort(['season', 'referee'])
)
refStats

```

season	referee	home_win_pct	home_pt_diff	...	foul_diff	experience	gender	games
str	str	f64	f64	...	f64	i64	str	i64
"2017-2018"	"Aaron Smith"	0.625	2.8	...	1.1	2	null	56
"2017-2018"	"Ben Taylor"	0.644823	5.606452	...	-0.73871	6	null	62
"2017-2018"	"Bennie Adams"	0.476615	0.5	...	0.176923	22	null	65
"2017-2018"	"Bill Kennedy"	0.588618	2.527941	...	1.289706	22	null	68
"2017-2018"	"Bill Spooner"	0.607588	2.690196	...	0.882353	28	null	51
...	...	...	...	...	...	...	...	...
"2025-2026"	"Tony Brothers"	0.621	3.9	...	-0.8	32	"MALE"	58
"2025-2026"	"Tre Maddox"	0.622222	5.044444	...	2.344444	16	"MALE"	45
"2025-2026"	"Tyler Ford"	0.592797	4.215254	...	-0.452542	12	"MALE"	59
"2025-2026"	"Tyler Ricks"	0.55	-1.5	...	0.8	5	"MALE"	40

season	referee	home_win_pct	home_pt_diff	...	foul_diff	experience	gender	games
str	str	f64	f64	...	f64	i64	str	i64
"2025-2026"	"Zach Zarba"	0.5667	1.526667	...	-0.826667	23	"MALE"	60

Cleaning Games File

```
games = pl.read_csv(here('Data/nba_games_ref_assignments.csv'))

games = (games
  ## Splitting season code (22025 --> game type 2, start year 2025)
  .with_columns(
    game_type = pl.col('season') // 10000,
    start_year = pl.col('season') % 10000,
  )
  ## Make it look like the referee table above ('2025-2026')
  .with_columns(
    season = pl.col('start_year').cast(pl.String) + '-' + (pl.col('start_year') + 1).cast(pl.String)
  )
  ## Keep only regular season games (2017-18)
  .filter(pl.col('game_type') == 2, pl.col('start_year') >= 2017)
  ## Fix team names
  .with_columns(
    pl.col(['home_team', 'away_team']).str.replace('L.A.', 'LA', literal=True)
  )
  ## Remove games listed twice
  .unique(subset = 'game_id', keep = 'first', maintain_order=True)
  ## Remove games missing referees
  .drop_nulls(subset = ['official1', 'official2', 'official3'])
  ## Drop 4th referee column (empty during reg season)
  .drop(['official4', 'official4_code', 'official4_JNum'])
)
games.shape
```

```
(10774, 21)
```

Matching Referee Stats to Games File - Fixing Names

```
## Referee Name fixed
nameFix = {
  'J.T. Orr': 'JT Orr',
  'Sharae Mitchell': 'ShaRae Mitchell', 'ShaRae Mitchell': 'ShaRae Mitchell',
  'Pat OConnell': 'Pat O'Connell', 'PAT OConnell': 'Pat O'Connell', 'PAT O'Connell': 'Pat O'Connell',
  'Cheryl Flores': 'Che Flores',
  'Jamahl Ralls': 'JD Ralls',
  'CP Painter': 'Chelisa Painter',
}

refStats = refStats.with_columns(pl.col('referee').replace(nameFix))

## One row per referee per game, attaching stats
gameRefs = (games
  .unpivot(index = ['game_id', 'season'], on = ['official1', 'official2', 'official3'],
    variable_name = 'slot', value_name = 'referee')
  .with_columns(pl.col('referee').replace(nameFix))
  .join(refStats, on = ['season', 'referee'], how = 'left')
)
```

```
## Check - should be 0
gameRefs['fouls_per_game'].null_count()
```

```
0
```

Average 3 Referees and Attach Games

```
crewStats = (gameRefs
    .group_by('game_id')
    .agg(
        crew_fouls_per_game = pl.col('fouls_per_game').mean(),
        crew_experience = pl.col('experience').mean(),
        crew_foul_diff = pl.col('foul_diff').mean(),
        crew_home_win_pct = pl.col('home_win_pct').mean(),
    )
)

## Attaching stats to games _TranslateTable
games = games.join(crewStats, on = 'game_id', how = 'left')
games.shape
```

```
(10774, 25)
```

For Code Below, We Don't Want Pull Data Every Time - Already Ran It Once

Pulling Each Season's Team Fouls

```
# import time
# from nba_api.stats.endpoints import leaguegamelog

# foulList = []
# for year in range(2017, 2026):
#     seasonCode = f'{year}-{str(year + 1)[-2:]}' ## nba_api wants '2017-18' not '2017-2018'

#     log = leaguegamelog.LeagueGameLog(
#         season = seasonCode,
#         season_type_all_star= 'Regular Season',
#         timeout = 60,
#     )

#     ## Turn result into polars table
#     result = log.get_dict()['resultSet'][0]
#     seasonLog = pl.DataFrame(result['rowSet'], schema = result['headers'], orient='row', infer_schema_length=None)
#     foulList.append(seasonLog.select('GAME_ID', 'TEAM_ABBREVIATION', 'PF', 'MIN'))
#     time.sleep(1) ## 1 second pause so we don't get blocked on NBA StopIteration

# ## Stack all seasons and save
# gameLogs = pl.concat(foulList)
# gameLogs.write_csv(here('Data/nba_team_game_logs.csv'))
# gameLogs
```

Reading Commented Out Above and Saving

```
gameLogs = pl.read_csv(here('Data/nba_team_game_logs.csv'))
gameLogs['MIN'].value_counts().sort('MIN')
```

MIN	count
i64	u32
240	20372
265	994
290	120
315	10
340	2

Total Fouls Per Game, Then Join

```
totalFouls = (gameLogs
  .with_columns(game_id = pl.col('GAME_ID').cast(pl.Int64)) ## 0021700001 --> 21700001
  .group_by('game_id')
  .agg(
    total_fouls = pl.col('PF').sum(),
    n_teams = pl.len(), ## about 2 every game
    game_minutes = pl.col('MIN').max(),
  )
)

## Attach to games
games = games.join(totalFouls, on = 'game_id', how = 'left')
games.shape
```

```
(10774, 28)
```

Checks

```
## Every game have exactly 2 teams
totalFouls['n_teams'].value_counts()
```

n_teams	count
u32	u32
2	10749

```
## How many games didn't get total_fouls
games['total_fouls'].null_count()
```

```
45
```

45 Missing Games (1/2 a Percent)

```
games.filter(pl.col('total_fouls').is_null()).group_by('season').len()
```

season	len
str	u32
"2019-2020"	45

Drop Games, and n_teams Column (Only Needed For One Check)

```
games = (games
          .drop_nulls(subset = 'total_fouls')
          .drop('n_teams')
        )
games.shape
```

```
(10729, 27)
```

The 45 missing games are from 2019-20 (COVID Season) - league shut down

We first tried the team fouls file from NBAstuffer, but it stops at 2024-25, so all 2025-26 games were blank. Instead, we calculated each team's season foul average ourselves from the game logs, which covers every season.

Team Season Avg From gameLogs

```
teamSeasonFouls = (gameLogs
                    ## Get season from game ID
                    .with_columns(start_year = 2000 + (pl.col('GAME_ID') // 100000) % 100)
                    .with_columns(
                        season = pl.col('start_year').cast(pl.String) + '-' + (pl.col('start_year') + 1).cast(pl.String)
                    )
                    ## Average fouls per game for each team in season
                    .group_by(['season', 'TEAM_ABBREVIATION'])
                    .agg(team_fouls = pl.col('PF').mean())
                  )
teamSeasonFouls
```

season	TEAM_ABBREVIATION	team_fouls
str	str	f64
"2018-2019"	"POR"	20.353659
"2018-2019"	"CHA"	18.902439
"2018-2019"	"IND"	19.439024
"2018-2019"	"PHX"	23.560976
"2019-2020"	"LAC"	22.138889
...	...	...
"2024-2025"	"GSW"	19.158537
"2025-2026"	"MIA"	18.865854
"2025-2026"	"MEM"	20.487805
"2025-2026"	"DET"	21.987805
"2025-2026"	"DEN"	19.134146

Attach Team Foul Averages (Home and Away)

```
games = (games
          .join(teamSeasonFouls.rename({'TEAM_ABBREVIATION': 'home_team_abbr', 'team_fouls': 'home_team_fouls'}), on =
            ['season', 'home_team_abbr'], how = 'left')
          .join(teamSeasonFouls.rename({'TEAM_ABBREVIATION': 'away_team_abbr', 'team_fouls': 'away_team_fouls'}), on =
            ['season', 'away_team_abbr'], how = 'left')
```

```
)
games.select(pl.col('home_team_fouls', 'away_team_fouls').null_count())
```

home_team_fouls	away_team_fouls
u32	u32
0	0

All files combined into one table (games). There's 10,729 games, each with crew stats, team foul avgs, and total_fouls (target number).

Removing Unuseful Columns

```
gamesClean = (games
    .with_columns(game_date = pl.col('game_date').str.to_date('%m/%d/%Y')) ## Changing date format
    ## Overtime periods and late season
    .with_columns(
        overtime = ((pl.col('game_minutes') - 240) / 25).cast(pl.Int64),
        days_left = (pl.col('game_date').max().over('season') - pl.col('game_date')).dt.total_days(),
    )
    .with_columns(late_season = pl.col('days_left') <= 14)
    .select(
        'game_id', 'game_date', 'season', 'home_team_abbr', 'away_team_abbr', 'official1', 'official2', 'official3',
        'crew_fouls_per_game', 'crew_experience', 'crew_foul_diff', 'crew_home_win_pct', 'home_team_fouls', 'away_team_fouls',
        'overtime', 'late_season', 'total_fouls' ## Only keeping useful columns
    )
    .sort('game_date')
)
gamesClean.write_parquet(here('Data/nba_games_clean.parquet'))
gamesClean
```

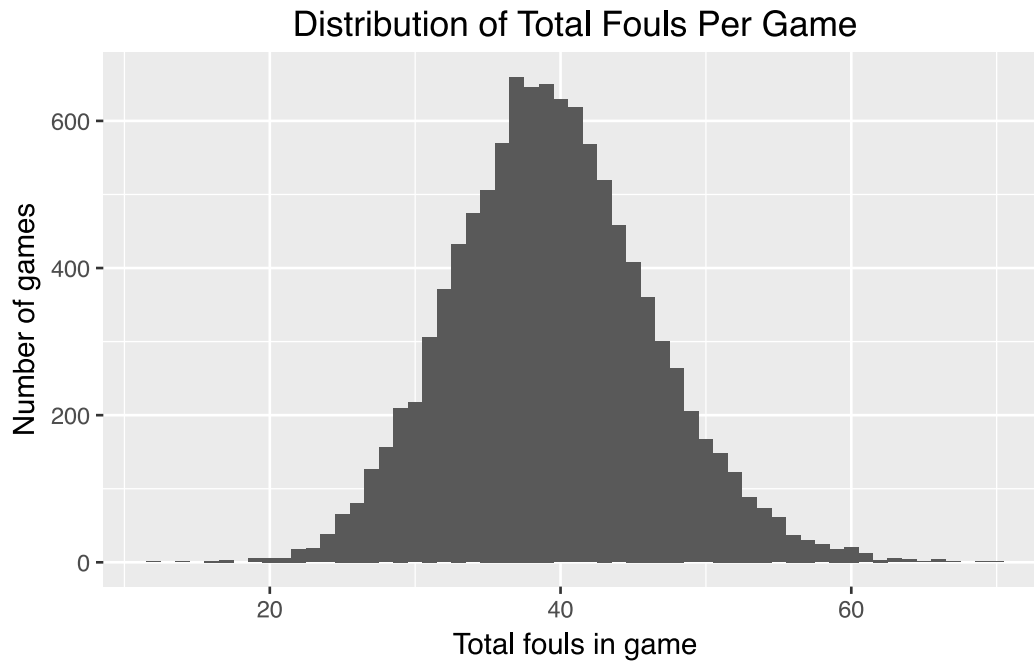
game_id	game_date	season	home_team_abbr	...	away_team_fouls	overtime	late_season	total_fouls
i64	date	str	str	...	f64	i64	bool	i64
21700001	2017-10-17	"2017-2018"	"CLE"	...	19.731707	0	false	49
21700002	2017-10-17	"2017-2018"	"GSW"	...	19.47561	0	false	41
21700003	2017-10-18	"2017-2018"	"DET"	...	17.182927	0	false	36
21700004	2017-10-18	"2017-2018"	"IND"	...	20.585366	0	false	50
21700005	2017-10-18	"2017-2018"	"ORL"	...	20.097561	0	false	38
...	...	...	...	...	...	...	...	...
22501196	2026-04-12	"2025-2026"	"OKC"	...	20.621951	0	true	17
22501197	2026-04-12	"2025-2026"	"SAS"	...	19.134146	0	true	34
22501198	2026-04-12	"2025-2026"	"LAL"	...	21.134146	0	true	34
22501199	2026-04-12	"2025-2026"	"LAC"	...	19.609756	0	true	45
22501200	2026-04-12	"2025-2026"	"POR"	...	19.682927	0	true	39

Exploratory Data Analysis (EDA)

Histogram of Total Fouls

```
from plotnine import *
import pyarrow
```

```
(ggplot(gamesClean, aes(x = 'total_fouls'))
  + geom_histogram(binwidth=1)
  + labs(x = 'Total fouls in game', y = 'Number of games', title = 'Distribution of Total Fouls Per Game')
)
```



Look at Unusual Games

```
gamesClean['total_fouls'].describe()
```

statistic	value
str	f64
"count"	10729.0
"null_count"	0.0
"mean"	39.452698
"std"	6.885991
"min"	12.0
"25%"	35.0
"50%"	39.0
"75%"	44.0
"max"	70.0

Games With Fewest and Most Fouls

```
## Lowest
gamesClean.sort('total_fouls').select('game_date', 'home_team_abbr', 'away_team_abbr', 'total_fouls').head(10)
```

game_date	home_team_abbr	away_team_abbr	total_fouls
date	str	str	i64
2024-04-09	"MIL"	"BOS"	12
2026-01-12	"IND"	"BOS"	14
2025-04-11	"PHX"	"SAS"	16
2025-04-13	"SAC"	"PHX"	16
2024-03-02	"LAL"	"DEN"	17
2026-02-19	"GSW"	"BOS"	17
2026-04-12	"OKC"	"PHX"	17
2024-04-09	"UTA"	"DEN"	19
2024-04-12	"DAL"	"DET"	19
2024-04-14	"CLE"	"CHA"	19

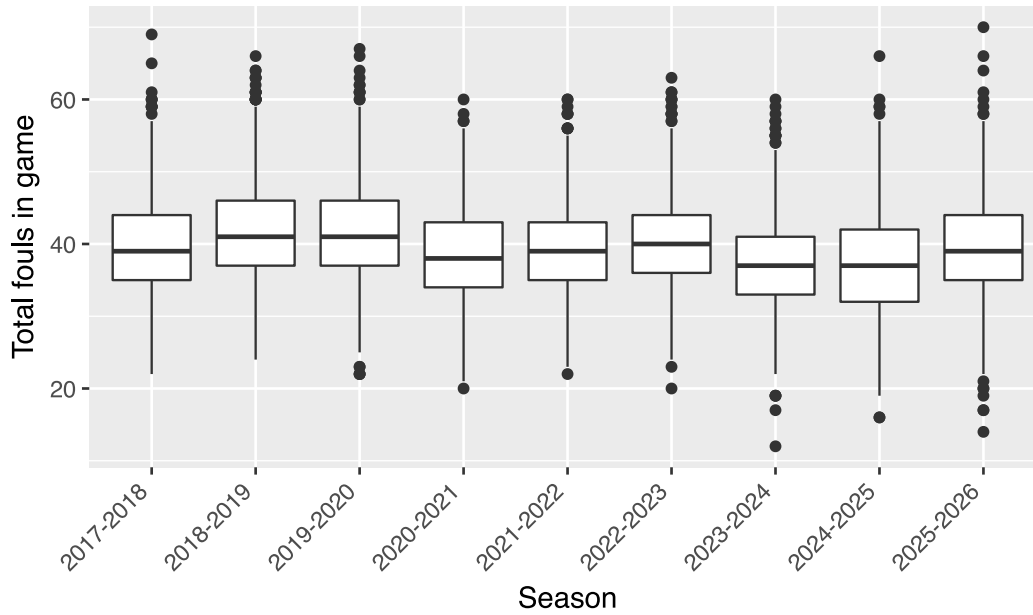
```
## Highest
gamesClean.sort('total_fouls',      descending=True).select('game_date',      'home_team_abbr',      'away_team_abbr',
'total_fouls').head(10)
```

game_date	home_team_abbr	away_team_abbr	total_fouls
date	str	str	i64
2025-10-23	"IND"	"OKC"	70
2017-11-29	"PHI"	"WAS"	69
2020-07-31	"DAL"	"HOU"	67
2018-11-09	"PHI"	"CHA"	66
2019-10-26	"PHX"	"LAC"	66
2024-10-25	"TOR"	"PHI"	66
2025-11-05	"LAL"	"SAS"	66
2017-12-31	"HOU"	"LAL"	65
2019-01-02	"PHX"	"PHI"	64
2019-03-01	"ATL"	"CHI"	64

Fouls Over Time

```
(ggplot(gamesClean, aes(x = 'season', y = 'total_fouls'))
  + geom_boxplot()
  + labs(x = 'Season', y = 'Total fouls in game', title = 'Total Fouls Per Game by Season')
  + theme(axis_text_x = element_text(rotation = 45, ha = 'right'))
)
```

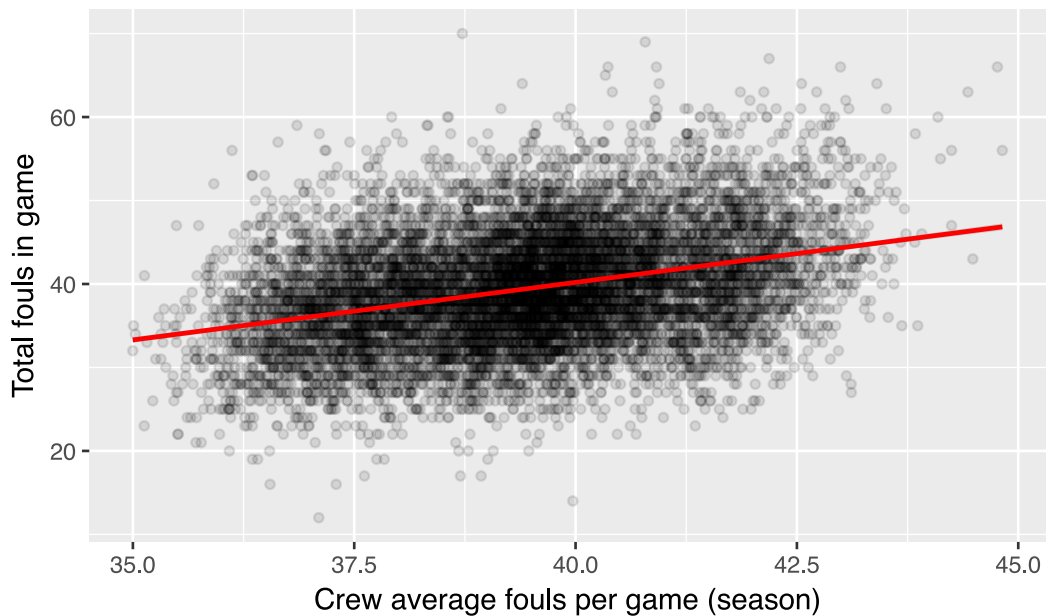
Total Fouls Per Game by Season



Does The Crew Actually Matter

```
(ggplot(gamesClean, aes(x = 'crew_fouls_per_game', y = 'total_fouls'))
+ geom_point(alpha = 0.1)
+ geom_smooth(method = 'lm', color = 'red')
+ labs(x = 'Crew average fouls per game (season)', y = 'Total fouls in game', title = 'Crew Foul Tendency vs.
Total Fouls')
)
```

Crew Foul Tendency vs. Total Fouls



Correlations

```
corrTable = (gamesClean
  .select('total_fouls', 'crew_fouls_per_game', 'crew_experience', 'crew_foul_diff',
    'crew_home_win_pct', 'home_team_fouls', 'away_team_fouls', 'overtime')
  .corr()
  .select(pl.all()).round(2))
## Short names so the table fits on the page
.rename({
  'total_fouls': 'total', 'crew_fouls_per_game': 'crew_fouls', 'crew_experience': 'crew_exp',
  'crew_foul_diff': 'crew_diff', 'crew_home_win_pct': 'crew_hwp', 'home_team_fouls': 'home_fouls',
  'away_team_fouls': 'away_fouls', 'overtime': 'OT',
})
)

## Add a label column so each row has a name too
## Show all columns for just this table
with pl.Config(tbl_cols = 10):
  display(corrTable.insert_column(0, pl.Series('variable', corrTable.columns)))
```

variable	total	crew_fouls	crew_exp	crew_diff	crew_hwp	home_fouls	away_fouls	OT
str	f64	f64	f64	f64	f64	f64	f64	f64
"total"	1.0	0.34	-0.04	0.05	0.05	0.3	0.29	0.21
"crew_fouls"	0.34	1.0	-0.12	0.19	0.24	0.48	0.48	0.04
"crew_exp"	-0.04	-0.12	1.0	0.04	0.01	-0.06	-0.07	-0.02
"crew_diff"	0.05	0.19	0.04	1.0	0.24	0.07	0.15	-0.01
"crew_hwp"	0.05	0.24	0.01	0.24	1.0	0.12	0.15	0.0
"home_fouls"	0.3	0.48	-0.06	0.07	0.12	1.0	0.25	0.02
"away_fouls"	0.29	0.48	-0.07	0.15	0.15	0.25	1.0	0.0
"OT"	0.21	0.04	-0.02	-0.01	0.0	0.02	0.0	1.0